

Distributed Systems

Docker & Local Kubernetes Project

Containerize, secure, publish and orchestrate an existing application

Master of Science - Data Engineering
Cohort: MSc DE1

Project Information

Module	Distributed Systems
Cohort	MSc DE1
Format	Individual project
Release date	14 September 2026
Project duration	8 calendar days - from 14 to 21 September 2026
Deadline	21 September 2026 before 23:59
Submission	Blackboard + public GitHub repository + Docker Hub image
Environment	Local workstation, Docker, Docker Hub and local Kubernetes cluster
Grading	/20

Project Goal

Your objective is to take an existing application that is **not containerized**, make sure it works locally, then design a complete containerization and local orchestration solution. You will create the **Docker files** yourself, build a secure image, publish it to **Docker Hub**, and deploy several replicas of the application to a **local Kubernetes cluster**.

The objective is not to redesign the application. The objective is to demonstrate a clean, secure and reproducible containerization and orchestration workflow.

Build it locally. Containerize it correctly. Secure it. Publish it. Orchestrate it.

1. Starter Application

For this project, you must use the following public **GitHub** application:

Required Starter Repository

UBC Flask Sample App

<https://github.com/ubc/flask-sample-app>

The starter project is a small **Python Flask REST API** with unit tests. It contains application code, tests, a dependency file and a local launch script. *At the time this project was prepared, the repository did not provide a **Dockerfile** in its main project structure.*

What you must do first

Before creating any **Docker** or **Kubernetes** file, you must:

1. **Fork or clone** the starter repository into your own **GitHub** account.
2. Read the existing `README.md` and inspect the source code.
3. Create a **Python** virtual environment.
4. Install the dependencies from `requirements.txt`.
5. Run the application **without Docker**.
6. Run the existing unit tests and verify that they pass.
7. Record the initial application behavior in your report.

Baseline requirement

You must prove that the original application works **before** containerization. If the application does not work on your machine, identify and document the cause before continuing.

Application routes to verify

At minimum, test the routes documented by the starter project, including:

- **GET** /
- **GET** /items
- **GET** /items/{item_id}
- **POST** /items

You may improve the application slightly if necessary for containerization, but the original functionality must remain available.

2. Required GitHub Repository

Your own public repository must contain the original application plus all files you create for the project.

Suggested Repository Name

msc-de1-distributed-systems-docker-k8s

Minimum expected structure

```
msc-de1-distributed-systems-docker-k8s/
|-- app/
|-- tests/
|-- Dockerfile
|-- .dockerignore
|-- compose.yaml
|-- requirements.txt
|-- run.py
|-- kind/
|   '-- kind-config.yaml
|-- k8s/
|   |-- namespace.yaml
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- network-policy.yaml
|   '-- optional-config-or-secret.yaml
|-- security/
|   |-- vulnerability-scan.txt
|   '-- sbom.*
|-- evidence/
|   '-- screenshots-or-command-output/
|-- README.md
'-- .gitignore
```

Git requirements

Your repository must show an understandable development history. You are expected to:

- create **meaningful commits** throughout the project;
- avoid a single final commit containing the entire solution;
- use clear commit messages;
- never commit passwords, tokens, credentials or private keys;
- keep temporary files, virtual environments and local secrets out of **Git** using **.gitignore**.

Academic integrity

The **Dockerfile**, Compose file, **Kubernetes manifests** and security configuration must be **your own work**. Copying a complete ready-made **Docker/Kubernetes** solution from another repository defeats the purpose of the project.

3. Dockerization Requirements

Part A - Build a Production-Oriented Docker Image

Create the complete Dockerization of the Flask application yourself.

Your repository must contain a **Dockerfile** that builds a functional image and starts the application correctly.

Mandatory Dockerfile requirements

Your Dockerfile must demonstrate the following practices:

1. Use an appropriate official **Python base image**.
2. Prefer a **small base image** such as a slim variant where appropriate.
3. Use a clear **WORKDIR**.
4. Copy dependency files before the application source when possible to improve layer caching.
5. Install only the dependencies required by the application.
6. Avoid unnecessary package-manager caches and temporary files.
7. Copy only the files required at runtime.
8. Run the application as a **non-root user**.
9. Expose only the application port that is required.
10. Use an appropriate executable **CMD** or **ENTRYPOINT** form.
11. Ensure the container can receive termination signals correctly.
12. Add a health check at **Docker** level if you can implement one without weakening the image.

Mandatory .dockerignore

Create a **.dockerignore** file. It must exclude unnecessary or sensitive content such as:

- **.git/** and **Git metadata**;
- **Python** virtual environments;
- **Python** caches;
- local **IDE files**;
- local environment files containing secrets;
- **test/output artifacts** that are not required in the runtime image;
- reports and screenshots that are unrelated to the application runtime.

Image objective

The image must be **reproducible, reasonably small, understandable and secure**. *Do not optimize image size by removing components that are necessary for correct execution.*

4. Local Container Execution and Compose

After building the image, prove that it works locally with **Docker**.

Required Docker verification

You must demonstrate:

- successful **docker build**;
- successful container start;
- access to the **Flask API** from the host;
- correct **API** behavior inside the container;
- application logs using `docker logs`;
- container **health/status inspection**;
- clean container stop and removal.

Required compose.yaml

Create a simple **Docker Compose** file for local execution. Even though the starter application is a single service, the file must provide a clean, reproducible way to start it.

Your Compose configuration must include, where relevant:

- the application service;
- the correct port mapping;
- a restart policy appropriate for local testing;
- environment variables without hard-coded secrets;
- the application health check;
- security-related options supported by Compose;
- a clear image name **and/or** build configuration.

Forbidden configuration

Do **not** use `privileged: true`. Do not mount the **Docker socket** into the application container. Do not use host networking unless you can justify it technically in the report.

Image inspection

Include evidence for at least the following:

- final image size;
- image layers using an inspection command such as `docker history`;
- container user identity proving that the application does not run as root;
- exposed ports and runtime configuration.

5. Container Security Requirements

Security is a mandatory part of the project and is evaluated independently from basic functionality.

Image and runtime security

Your solution must address all of the following:

1. **Non-root execution:** the application process must not run as root.
2. **Minimal image:** do not include development tools that are unnecessary at runtime.
3. **Dependency hygiene:** use the existing dependency file and document any dependency changes.
4. **No secrets in images:** no credentials in **Dockerfile** instructions, image layers or committed environment files.
5. **Reduced privileges:** avoid privileged mode and unnecessary **Linux capabilities**.
6. **Read-only approach:** use a read-only root filesystem where compatible with the application, or explain why it is not used.
7. **Resource awareness:** do not allow the application to consume unlimited resources in **Kubernetes**.

Vulnerability scan

Scan the final image with a recognized tool such as **Trivy** or **Docker Scout**. Save the result in the `security/` directory.

Your report must state:

- the tool used;
- the image tag scanned;
- the number of findings by severity;
- what you changed after the scan;
- any remaining **HIGH** or **CRITICAL** finding and why it remains.

Software Bill of Materials

Generate an **SBOM** for the final image using a tool such as *Syft*, *Docker Scout SBOM*, or another equivalent tool. Store the output in `security/` using either **SPDX** or **CycloneDX** format.

Security grading principle

You are evaluated on your ability to **identify, reduce and explain risk**. A scan with findings is not automatically a failure; ignoring the findings without analysis is.

6. Docker Hub Publication

Create a public repository on **Docker Hub** for your final image.

Required image tags

Publish at least:

- a **semantic/versioned** tag such as **1.0.0**;
- the **latest** tag.

The image name should follow a clear convention, for example:

```
<dockerhub-username>/msc-de1-flask-app:1.0.0
```

Required verification

After pushing the image:

1. remove or ignore your local build cache as appropriate;
2. pull the published image from **Docker Hub**;
3. run the pulled image locally;
4. verify that the **API still works**;
5. add the public **Docker Hub URL** to your **GitHub README**.

Traceability requirement

The **Docker Hub** image used by **Kubernetes** must correspond to the version documented in your **GitHub repository**. Your **README** must clearly identify the image and tag used for the final deployment.

7. Local Kubernetes Cluster

You must deploy the **Docker Hub** image to a **local Kubernetes cluster**. For this project, the recommended tool is **kind** (**Kubernetes IN Docker**).

Required cluster topology

Create a small multi-node cluster with:

- 1 control-plane node;
- 2 worker nodes.

Store the cluster definition in:

```
kind/kind-config.yaml
```

Required Kubernetes objects

Your `k8s/` folder must contain valid **YAML manifests** for at least:

1. **Namespace**
2. **Deployment**
3. **Service**
4. **NetworkPolicy**

Use a dedicated namespace such as:

```
msc-de1-project
```

Deployment requirements

The Deployment must include:

- at least **2 replicas**;
- the image pulled from your **Docker Hub** repository;
- explicit container port configuration;
- **readiness probe**;
- **liveness probe**;
- **CPU** and memory **requests**;
- **CPU** and memory **limits**;
- a controlled rolling update strategy;
- **Kubernetes** labels and selectors that are clear and consistent.

You may use port-forwarding for local access. An Ingress is optional unless you choose to configure one correctly.

8. Kubernetes Security

Your **Kubernetes manifests** must not simply run the container with default privileges.

Required pod/container security context

Configure the Deployment to address the following:

- **runAsNonRoot: true;**
- a **non-root user/group** compatible with your image;
- **allowPrivilegeEscalation: false;**
- drop unnecessary **Linux capabilities**, preferably **ALL**;
- use **seccompProfile: RuntimeDefault**;
- use **readOnlyRootFilesystem: true** when compatible, or document a justified exception;
- avoid **hostPath** volumes, host networking and host **PID/IPC** access.

Configuration and secrets

If you add non-sensitive configuration, use a **ConfigMap** rather than hard-coding values in the image.

If you introduce a secret during your implementation:

- use a **Kubernetes Secret**;
- never commit the real secret value to **GitHub**;
- provide only a safe template or documented creation command.

Network isolation

Provide a **NetworkPolicy** that documents the intended network access to the application pods. *If the networking implementation of your local cluster does not enforce the policy by default, explain this limitation and optionally install a compatible CNI such as **Calico** or **Cilium** for enforcement.*

Security consistency

The security configuration must be compatible across the **Docker image**, **Compose execution** and **Kubernetes deployment**. A **Kubernetes non-root policy** cannot work if your image itself only works as root.

9. Distributed Systems Demonstrations

You must demonstrate that **Kubernetes** is doing more than starting one container.

A. Replication and service discovery

Show that:

- at least two application pods are running;
- the Service selects the correct pods;
- the application is accessible through the **Kubernetes Service** or port-forwarding;
- the pods may run on different worker nodes when scheduling permits.

B. Self-healing

Delete one running application pod manually and demonstrate that the **Deployment** automatically creates a replacement pod.

Your report must include the commands used and a short explanation of **desired state reconciliation**.

C. Scaling

Scale **the Deployment** from **2 replicas to 3 replicas**, verify the new state, then return to your chosen final replica count.

D. Rolling update and rollback

Create a small visible or version-related change to the application or image, publish a new version tag, then:

1. perform a **rolling update**;
2. inspect rollout status;
3. inspect rollout history;
4. demonstrate a **rollback** to the previous revision.

What is being assessed

You must be able to explain the difference between a **Docker container** and a **Kubernetes-managed application**: replication, desired state, health checking, self-healing, service abstraction and controlled updates.

10. Documentation Requirements

GitHub README

Your **README.md** must be sufficient for another student or evaluator to reproduce your work. It must contain:

1. project objective and architecture overview;
2. link to the original starter application;
3. prerequisites;
4. how to run the original application locally;
5. how to build and run the **Docker image**;
6. how to run the project with **Docker Compose**;
7. link to your public **Docker Hub repository**;
8. how to create the kind cluster;
9. how to deploy the **Kubernetes manifests**;
10. how to access and test the application;
11. how to **delete/clean** the local cluster;
12. security decisions and known limitations.

Short technical report - 4 to 6 pages

Submit a PDF report containing:

- **1. Baseline:** proof that the original app worked before **Docker**;
- **2. Dockerization:** important **Dockerfile** choices and image size;
- **3. Security:** non-root execution, scan findings and **SBOM**;
- **4. Docker Hub:** image repository and published tags;
- **5. Kubernetes:** cluster architecture and deployed objects;
- **6. Distributed behavior:** self-healing, scaling, rolling update and rollback;
- **7. Conclusion:** what you learned and one improvement you would make for a production deployment.

Screenshots should support technical explanations, not replace them. Command output may be included as evidence when it is readable and relevant.

11. Grading - 20 Points

Baseline application, Git repository and reproducibility	2 pts
Dockerfile quality, image build and local execution	4 pts
Docker/Container security, vulnerability scan and SBOM	3 pts
Docker Compose and Docker Hub publication	2 pts
Local kind cluster and Kubernetes manifests	4 pts
Kubernetes security, probes and resource controls	2 pts
Self-healing, scaling, rolling update and rollback demonstrations	2 pts
README and short technical report	1 pt
Total	20 pts

Major deductions

Significant deductions may apply when:

- the application does not run;
- the submitted image cannot be pulled from **Docker Hub**;
- the image runs as root without a valid justification;
- the **Kubernetes Deployment** cannot start the published image;
- credentials or secrets are committed to **GitHub**;
- **Docker/Kubernetes files** are copied from another complete solution without understanding or adaptation;
- required evidence cannot be reproduced from the repository documentation.

12. Blackboard Submission

Before **21 September 2026 at 23:59**, submit to **Blackboard**:

1. your **public GitHub repository URL**;
2. your **public Docker Hub repository URL**;
3. your **technical report PDF**;
4. optionally, a short text file containing both **URLs** if **Blackboard** requires a file upload.

Final verification before submission

Before submitting, test the project from a clean state as far as possible: clone the repository, follow the **README**, pull the **Docker Hub image**, create the local cluster, apply the manifests and verify that the application becomes reachable.

End of project document.

Deadline: 21 September 2026 before 23:59.